

JANE RIVEROS

HERMES VIDEO EDIT PLAYBOOK

EL PLAYBOOK DE EDICIÓN DE VIDEO

One raw take. One AI agent. A finished video.

Una toma en crudo. Un agente de IA. Un video terminado.

This guide is in **two languages** — use the contents below or scroll.

Esta guía está en **dos idiomas** — usa el índice o desplázate.

TABLE OF CONTENTS · ÍNDICE

❶ ENGLISH · SECCIÓN EN INGLÉS

· How to run it (Starter prompt) · The stack · Phase 1–5 · Gotchas · The shortcut

❷ ESPAÑOL · SECCIÓN EN ESPAÑOL

· Prompt inicial · El stack · Fase 1–5 · Obstáculos · El atajo

SECTION 1 · ENGLISH

ENGLISH

The Video Edit Playbook — free tools only

scroll for the full guide · desplázate para la guía completa

JANE RIVEROS

HERMES THE VIDEO EDIT PLAYBOOK

One raw take. One AI agent. A finished video.

The exact playbook a Hermes agent follows to turn a single talking-head video into an edited, captioned, silence-trimmed video: word-synced captions, green/red brand cards, sound design, and a music bed. Built with **free tools only** — nothing to pay for, nothing to install that costs money.

STRAIGHT FROM THE FINISHED EDIT

What you get out

Your raw talking-head video, edited underneath with its own audio. On top: a designed card for key lines, karaoke-style word captions, silence gaps removed, a progress bar, and a small royalty-free sound kit placed under the voiceover.

FIVE

phases, start to finish

100%

free tools

0

paid subscriptions

9:16

vertical MP4 output



No ElevenLabs. No paid API keys. No cloud credits. Everything below works with free, local, open-source tools.

How to run it

You don't run the commands. Your **Hermes agent** does. You supply three things: the raw video, a hex accent color (or "use brand orange" if you want), and taste. The agent does the rest in five phases.

PASTE THE WHOLE PLAYBOOK

COPY AND PASTE THE ENTIRE GUIDE FROM HERE AND BELOW

→ Select everything below, copy it, and paste it into your Hermes agent. That's the whole method in one block.

HOW TO RUN IT

You don't run the commands. Your Hermes agent does. You supply three things: the raw video, a hex accent color (or "use brand orange" if you want) and taste. The agent does the rest in five phases.

STARTER PROMPT (paste as-is)

Here's a raw talking-head video ([video.mp4]) and I want a vertical social edit. Follow the Hermes Video Edit Playbook:

- 1) transcribe the voiceover with word-level timestamps (free, local whisper),
 - 2) detect and remove the long silence gaps,
 - 3) add big word-synced captions + 4-6 brand cards (hex accent #FF4D00, dark background #0A0C0F, green #58e58c for wins, red #e5484d for costs),
 - 4) add a free sound kit + a subtle music bed (all generated locally, no paid tools),
 - 5) render a 1080x1920 MP4, then save a FINAL and an EDITABLE copy.
- Keep every card clear of my face. Lint, snapshot key beats and LOOK, fix anything broken, then render. Ask me only about layout taste and hook wording. Ship a reviewable draft.

THE STACK (all free)

- Hermes: the agent that runs the playbook (hermes app)
- ffmpeg / ffprobe: probe, silences, crops, audio, verify (brew install ffmpeg)
- whisper.cpp: FREE local transcription with word timestamps, no API (brew install whisper-cpp, model ggml-small)
- HyperFrames: render video from HTML (npx hyperframes, no key)
- GSAP: animation runtime (bundled locally, never CDN)
- ffmpeg SFX: generates the sound kit for free, no service

PHASE 1 · PROBE & TRANSCRIBE

```
ffprobe -v error -show_entries format=duration -show_entries stream=codec_name,width,height,r_frame_rate [video.mp4]
```

```
ffmpeg -i [video.mp4] -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav
```

```
whisper-cli -m ggml-small.bin -f audio.wav -ojf -otxt -of transcript
```

The -ojf JSON gives per-word timings; merge sub-word tokens into whole words.

PHASE 2 · SILENCE REMOVAL

- 1) Check audio health first:

```
ffmpeg -i [video.mp4] -af "volumedetect" -vn -f null - 2>&1 | grep -E "mean_volume|max_volume"
```

If mean_volume is below ~60 dB the audio is effectively empty – trim nothing and flag it.

- 2) Pick the threshold ~10 dB above mean_volume (voice ~-20/-25 dB → -30/-35 dB).

```
3) ffmpeg -i [video.mp4] -af "silencedetect=noise=-30dB:d=0.5" -f null - 2>&1 | grep silence_
```

- 4) Concat ONLY the keep-segments with ffmpeg concat (audio+video in sync).

Trim only real dead air (>~0.5s). Don't splice every comma (jump-cuts on the speaker). If there are no long silences, trimming nothing is correct.

PHASE 3 · CAPTIONS & BRAND CARDS

One HTML file: video + caption chunks + 4-6 cards + one paused GSAP timeline (no setTimeout, no infinite loops).

Tokens: --accent:#FF4D00; --bg:#0A0C0F; --win:#58e58c; --cost:#e5484d;

Captions: 1-3 words per chunk; show chunk.start-0.04 through the NEXT chunk's start-0.04.

Cards: hook, model/feature chips, before/after (red/green), cost (4 bills → 1), "no lock-in" badge, CTA ("comment [word]").

CRITICAL: keep every card clear of the face (below the name tag, above the chin, or the lower third). Make cards BIG (~2.5x on a 1080 canvas) and WELL-SPACED.

PHASE 4 · SOUND (FREE)

```
pop.wav -> ffmpeg -f lavfi -i "sine=frequency=1200:duration=0.15:beep_factor=2" pop.wav
```

```
tick.wav -> ffmpeg -f lavfi -i "sine=frequency=800:duration=0.06:s=44100" tick.wav
```

```
whoosh.wav -> ffmpeg -f lavfi -i "sine=frequency=300:duration=1" -af
```

```
"afade=t=in:st=0:d=0.3,afade=t=out:st=0.6:d=0.4" whoosh.wav
```

Place hits on key words at low volume (~0.3). Voice is the star. Optional music bed at -18/-20 dB under voice. No ElevenLabs, all local.

PHASE 5 · QA, RENDER, DELIVER

`npx hyperframes lint [folder]`

`npx hyperframes snapshot [folder] --at 2.7,12.2,27.8` # and LOOK at the frames

`npx hyperframes render [folder] -o output.mp4 --fps 30`

`ffprobe -v error -show_entries format=duration output.mp4` # verify the FILE, not the log

Save a FINAL and an EDITABLE copy.

GOTCHAS (read before you render)

- Frozen frame: every video/audio element needs a unique id.
- Frozen when seeking: re-encode with a keyframe every second (`-g 30 -keyint_min 30`).
- Element invisible at first appearance: set the hidden state in CSS, not a GSAP set at `t=0`.
- Mashing captions: end each chunk at the next chunk's start minus 0.04s.
- Overlay covering the face: keep cards in the clear band.
- Overlays too small/bunched: ~2.5x on 1080 and big gaps.
- Wrong duration: verify with `ffprobe` on the FILE.
- Word-level captions: use `whisper -ojf` and merge tokens.

THE SHORTCUT

Ask Hermes to save these five phases as a skill. Then next time just say:

"run the video edit playbook on [new-video.mp4] with accent #FF4D00 and make a final + editable copy"

PASTE THE WHOLE PLAYBOOK

COPY AND PASTE THE ENTIRE GUIDE FROM HERE AND BELOW

→ Select everything below, copy it, and paste it into your Hermes agent. That's the whole method in one block.

HOW TO RUN IT

You don't run the commands. Your Hermes agent does. You supply three things: the raw video, a hex accent color (or "use brand orange" if you want) and taste. The agent does the rest in five phases.

STARTER PROMPT (paste as-is)

Here's a raw talking-head video ([video.mp4]) and I want a vertical social edit. Follow the Hermes Video Edit Playbook:

- 1) transcribe the voiceover with word-level timestamps (free, local whisper),
 - 2) detect and remove the long silence gaps,
 - 3) add big word-synced captions + 4-6 brand cards (hex accent #FF4D00, dark background #0A0C0F, green #58e58c for wins, red #e5484d for costs),
 - 4) add a free sound kit + a subtle music bed (all generated locally, no paid tools),
 - 5) render a 1080x1920 MP4, then save a FINAL and an EDITABLE copy.
- Keep every card clear of my face. Lint, snapshot key beats and LOOK, fix anything broken, then render. Ask me only about layout taste and hook wording. Ship a reviewable draft.

THE STACK (all free)

- Hermes: the agent that runs the playbook (hermes app)
- ffmpeg / ffprobe: probe, silences, crops, audio, verify (brew install ffmpeg)
- whisper.cpp: FREE local transcription with word timestamps, no API (brew install whisper-cpp, model ggml-small)
- HyperFrames: render video from HTML (npx hyperframes, no key)
- GSAP: animation runtime (bundled locally, never CDN)
- ffmpeg SFX: generates the sound kit for free, no service

PHASE 1 · PROBE & TRANSCRIBE

```
ffprobe -v error -show_entries format=duration -show_entries stream=codec_name,width,height,r_frame_rate [video.mp4]
ffmpeg -i [video.mp4] -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav
whisper-cli -m ggml-small.bin -f audio.wav -ojf -otxt -of transcript
The -ojf JSON gives per-word timings; merge sub-word tokens into whole words.
```

PHASE 2 · SILENCE REMOVAL

- 1) Check audio health first:

```
ffmpeg -i [video.mp4] -af "volumedetect" -vn -f null - 2>&1 | grep -E "mean_volume|max_volume"
```

If mean_volume is below ~-60 dB the audio is effectively empty – trim nothing and flag it.
 - 2) Pick the threshold ~10 dB above mean_volume (voice ~-20/-25 dB → -30/-35 dB).
 - 3)

```
ffmpeg -i [video.mp4] -af "silencedetect=noise=-30dB:d=0.5" -f null - 2>&1 | grep silence_
```
 - 4) Concat ONLY the keep-segments with ffmpeg concat (audio+video in sync).
- Trim only real dead air (>=0.5s). Don't splice every comma (jump-cuts on the speaker). If there are no long silences, trimming nothing is correct.

PHASE 3 · CAPTIONS & BRAND CARDS

One HTML file: video + caption chunks + 4-6 cards + one paused GSAP timeline (no setTimeout, no infinite loops).

Tokens: --accent:#FF4D00; --bg:#0A0C0F; --win:#58e58c; --cost:#e5484d;

Captions: 1-3 words per chunk; show chunk.start=0.04 through the NEXT chunk's start=0.04.

Cards: hook, model/feature chips, before/after (red/green), cost (4 bills → 1), "no lock-in" badge, CTA ("comment [word]").

CRITICAL: keep every card clear of the face (below the name tag, above the chin, or the lower third). Make cards BIG (~2.5x on a 1080 canvas) and WELL-SPACED.

PHASE 4 · SOUND (FREE)

```
pop.wav    -> ffmpeg -f lavfi -i "sine=frequency=1200:duration=0.15:beep_factor=2" pop.wav
tick.wav   -> ffmpeg -f lavfi -i "sine=frequency=800:duration=0.06:s=44100" tick.wav
whoosh.wav -> ffmpeg -f lavfi -i "sine=frequency=300:duration=1" -af
"afade=t=in:st=0:d=0.3,afade=t=out:st=0.6:d=0.4" whoosh.wav
```

Place hits on key words at low volume (~0.3). Voice is the star. Optional music bed at -18/-20 dB under voice. No ElevenLabs, all local.

PHASE 5 · QA, RENDER, DELIVER

```
npx hyperframes lint [folder]
npx hyperframes snapshot [folder] --at 2.7,12.2,27.8 # and LOOK at the frames
npx hyperframes render [folder] -o output.mp4 --fps 30
```

PASTE THIS TO START

STARTER PROMPT

Here's a raw talking-head video ([video.mp4]) and I want an edit for vertical social. Follow the Hermes Video Edit Playbook: 1) transcribe the voiceover with word-level timestamps (free, local whisper), 2) detect and remove the long silence gaps, 3) add big word-synced captions + 4-6 brand cards (hex accent #FF4D00, dark background #0A0C0F, green #58e58c for wins, red #e5484d for costs), 4) add a free sound kit + a subtle music bed (all generated locally, no paid tools), 5) render a 1080x1920 MP4, then save a FINAL and an EDITABLE copy. Keep every card clear of my face. Lint, snapshot key beats and LOOK, fix anything broken, then render. Ask me only about layout taste and hook wording. Ship a reviewable draft.

That's the whole start. The five phases below are what the agent actually does — each one is a checkable, repeatable step.

EVERY TOOL + WHERE TO GET IT · ALL FREE

THE STACK

TOOL	WHAT IT DOES	GET IT
Hermes	The AI agent that runs this whole playbook end to end	hermes (app)
ffmpeg / ffprobe	Probe, silence-detect, crops, audio, and verifying output	brew install ffmpeg
whisper.cpp	FREE local speech-to-text with word timestamps (no API)	brew install whisper-cpp (model ggml-small)
HyperFrames	Renders video from HTML: captions, GSAP, snapshot, render	npx hyperframes (no key)
GSAP	Animation runtime inside the composition	bundled locally, never CDN
ffmpeg SFX	Generate the sound kit (pop, whoosh, ticks) for free	built into ffmpeg, no service

Every tool is free and runs locally. Nothing leaves your machine except the final MP4. If it needs a paid key, it's not in this playbook.

PHASE 1 · PROBE & TRANSCRIBE

Know your video, get word timestamps

Two things drive the entire edit: the exact duration, and the exact timing of every word. Both are free.

```
# duration + resolution + fps
ffprobe -v error -show_entries format=duration \
  -show_entries stream=codec_name,width,height,r_frame_rate source.mp4

# a clean mono 16kHz audio for transcription
ffmpeg -i source.mp4 -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav

# free local transcription WITH word timestamps (no API)
whisper-cli -m ggml-small.bin -f audio.wav -ojf -otxt -of transcript
```

The `-ojf` JSON gives per-token word timings inside each segment — that's what lets captions land on the exact word. (Some whisper versions output word times per token; your agent can merge sub-word tokens into whole words.)

PHASE 2 · SILENCE DETECTION & REMOVAL

Cut the dead air

Detect where the voice is silent, then reassemble only the speaking segments so the video is tight — no long gaps.

```
# find silence gaps (report where speech pauses >=0.5s)
ffmpeg -i source.mp4 -af "volumedetect" -vn -f null - 2>&1 | grep -E "mean_volume|max_volume"
# ^ FIRST: read the mean_volume. Pick your silence threshold about
#   10 dB above it (voice usually sits around -20 to -25 dB → use -30 to -35 dB)

ffmpeg -i source.mp4 -af "silencedetect=noise=-30dB:d=0.5" -f null - 2>&1 \
  | grep silence_

# then concat ONLY the keep-segments back together (agent stitches
# them with ffmpeg concat filter so audio+video stay in sync)
```

Living people have natural short pauses. Don't cut dead air (a beat or more) — don't splice every comma or you get jumps/cuts on the speaker.

A real silent / video-corrupted source (mean volume around -40 dB) has NO audible voice, so ANY threshold will flag it all as "silence." Check mean_volume first. If it's below about -30 dB, the file's audio is effectively empty — trim nothing and flag it to the user.

If a video has no long silences (deep energy, constant talking), skip the splice entirely — trimming nothing is also correct. On real voice (~-22 dB mean), -30 dB correctly finds only genuine dead air.

Build the composition

A single HTML file holds the video + caption chunk + 4-6 brand cards + one paused GSAP timeline. Keep cards clear of the face (below the name tag, above the chin / or in the lower third).

```
# accent tokens every card inherits
:root { --accent:#FF4D00; --bg:#0A0C0F; --win:#58e58c; --cost:#e5484d; }

# caption rule: 1-3 words per chunk; show chunk.start-0.04
# through the NEXT chunk's start-0.04 (else they mash together)
```

A proven minimal card set

- **Hook card** — first line, big all-caps word
- **Before / After** — red vs green
- **"No lock-in" badge** — green outline
- **Model/feature chips** — list key names
- **Cost card** — 4 bills → 1, red to green
- **CTA** — "comment [word]" + send-guide line

Every number gets a count-up. Every list pops word-synced. Rules the agent can follow every time.

PHASE 4 · SOUND DESIGN (FREE)

A 6-sound kit, generated by ffmpeg

No ElevenLabs. ffmpeg synthesizes tiny royalty-free hits, then you place them on key words — a whoosh on a card enter, ticks under a count-up, a pop on a chip.

```
# generate the kit with ffmpeg (free, local) — all commands regression-tested
# pop: a short punchy beep
ffmpeg -f lavfi -i "sine=frequency=1200:duration=0.15:beep_factor=2" pop.wav
# tick: a tiny high click
ffmpeg -f lavfi -i "sine=frequency=800:duration=0.06:s=44100" tick.wav
# whoosh: a rising-then-fading sweep (fade in/out = the "whoosh" shape)
ffmpeg -f lavfi -i "sine=frequency=300:duration=1" -af \
    "afade=t=in:st=0:d=0.3,afade=t=out:st=0.6:d=0.4" whoosh.wav
```

For a music bed, generate a soft ambient pad with ffmpeg too, OR add any royalty-free bed you own. Keep it -18 to -20 dB under the voice so speech stays clear.

Mix balance sanity: place hits at low volume (-1-3). The voice is the star. OFX should lead and fade under it.

PHASE 5 · QA, RENDER, DELIVER

Look at pixels. Then ship.

Every bug is caught by looking at frames, never by re-reading code. Lint, snapshot, LOOK, fix, render, then verify the file.

```
# lint + snapshot key beats + render
npx hyperframes lint
npx hyperframes snapshot --at 2.7,12.2,27.8
npx hyperframes render -o output.mp4 --fps 30

# verify the FILE (never trust the render log)
ffprobe -v error -show_entries format=duration output.mp4
```

Save a FINAL and an EDITABLE copy — so any tweak is a regenerate, not a redo.

READ BEFORE YOUR FIRST RENDER

The gotchas

SYMPTOM	FIX
Video renders as a frozen frame	Every video/audio element needs a unique id.
Frozen frame when video moves/seeks	Re-encode with a keyframe every second (<code>-g 30 -keyint_min 30</code>).
Element invisible at first appearance	Set initial hidden state in CSS, not in a GSAP set at time 0.
Two captions mashed together	End each chunk at the next chunk's start minus 0.04s.
Overlays covering the speaker's face	Keep cards in the clear band — below the name tag, above the chin, or the lower third.
Overlays too small / bunched	Size to ~2.5x on a 1080 screen and space big gaps between elements.
Wrong duration	Verify with <code>ffprobe</code> on the FILE, not the render log.
Need word-level captions	Use <code>whisper -ojf</code> and merge grouped word tokens for accurate timing.

THE SHORTCUT · ONE SKILL, INSTALLED

Save it as a Hermes skill

Ask Hermes to save this five-phase playbook as a reusable **skill**. Then next time you only say:

```
run the video edit playbook on [new-video.mp4] with accent #FF4D00 and make a final + editable copy
```

WHY FREE & LOCAL

No monthly fees, no credit limits, no key management. ffmpeg + whisper + HyperFrames are open source and run on your machine.

WHY IT WORKS EVERY TIME

Every step is checkable and deterministic: lint, snapshot frames, LOOK, fix. The agent never renders blind.

SECCIÓN 2 · ESPAÑOL

ESPAÑOL

El Playbook de Edición de Video — solo herramientas gratuitas

guía completa en español

JANE RIVEROS

HERMES EL PLAYBOOK DE EDICIÓN DE VIDEO

Una toma en crudo. Un agente de IA. Un video terminado.

El playbook exacto que un agente de Hermes sigue para convertir una sola toma de video hablando a cámara en un video editado, con subtítulos, sin silencios: captions sincronizados palabra por palabra, tarjetas de marca en verde/rojo, diseño de sonido y una cama musical. Construido solo con **herramientas gratuitas** — sin pagar nada, sin instalar nada que cueste dinero.

DIRECTO DESDE LA EDICIÓN TERMINADA

Qué obtienes

Tu video en crudo hablando a cámara, editado debajo con su propio audio. Encima: una tarjeta diseñada para las frases clave, captions de palabras tipo karaoke, silencios eliminados, una barra de progreso y un pequeño kit de sonido libre de regalías bajo la voz.

CINCO

fases, de principio a fin

100%

herramientas gratuitas

0

suscripciones de pago

9:16

salida MP4 vertical



Sin ElevenLabs. Sin claves API de pago. Sin créditos en la nube. Todo lo que sigue funciona con herramientas gratuitas, locales y de código abierto.

Cómo ejecutarlo

Tú no ejecutas los comandos. Tu **agente de Hermes** los ejecuta. Tú aportas tres cosas: el video en crudo, un color de acento hex (o "usa naranja de marca" si quieres) y el gusto. El agente hace el resto en cinco fases.

PEGA TODO EL PLAYBOOK

COPIA Y PEGA TODO EL PLAYBOOK DESDE AQUÍ Y MÁS ABAJO

→ Selecciona todo lo de abajo, cópialo y pégalo en tu agente de Hermes. Ese es todo el método en un solo bloque.

CÓMO EJECUTARLO

Tú no ejecutas los comandos. Tu agente de Hermes los ejecuta. Tú aportas tres cosas: el video en crudo, un color de acento hex (o "usa naranja de marca" si quieres) y el gusto. El agente hace el resto en cinco fases.

PROMPT INICIAL (pégalo tal cual)

Aquí hay un video en crudo hablando a cámara ([video.mp4]) y quiero una edición para redes vertical. Sigue el Playbook de Edición de Video de Hermes:

- 1) transcribe la voz con tiempos por palabra (whisper local, gratis),
- 2) detecta y elimina los huecos largos de silencio,
- 3) añade captions grandes sincronizados + 4-6 tarjetas de marca (acento hex #FF4D00, fondo oscuro #0A0C0F, verde #58e58c para logros, rojo #e5484d para costos),
- 4) añade un kit de sonido gratis + una cama musical sutil (todo generado localmente, sin herramientas de pago),
- 5) renderiza un MP4 1080x1920 y guarda una copia FINAL y una EDITABLE.

Mantén cada tarjeta lejos de mi rostro. Lint, snapshot de hitos clave y MIRA, corrige lo que esté roto y luego renderiza. Pregúntame solo por el gusto del diseño y la palabra del gancho. Entrega un borrador revisable.

EL STACK (todo gratis)

- Hermes: el agente que ejecuta el playbook (app hermes)
- ffmpeg / ffprobe: sondeo, silencios, recortes, audio, verificación (brew install ffmpeg)
- whisper.cpp: transcripción local GRATIS con tiempos por palabra, sin API (brew install whisper-cpp, modelo ggml-small)
- HyperFrames: render desde HTML (npx hyperframes, sin clave)
- GSAP: motor de animación (empaquetado local, nunca CDN)
- ffmpeg SFX: genera el kit de sonido gratis, sin servicio

FASE 1 · SONDEO Y TRANSCRIPCIÓN

```
ffprobe -v error -show_entries format=duration -show_entries stream=codec_name,width,height,r_frame_rate [video.mp4]
```

```
ffmpeg -i [video.mp4] -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav
```

```
whisper-cli -m ggml-small.bin -f audio.wav -ojf -otxt -of transcript
```

El JSON -ojf da tiempos por palabra; une los tokens de sub-palabra en palabras completas.

FASE 2 · DETECCIÓN Y ELIMINACIÓN DE SILENCIOS

- 1) Revisa la salud del audio primero:

```
ffmpeg -i [video.mp4] -af "volumedetector" -vn -f null - 2>&1 | grep -E "mean_volume|max_volume"
```

Si mean_volume está por debajo de ~-60 dB, el audio está vacío - no recortes nada y avisa.

- 2) Elige el umbral ~10 dB por encima del mean_volume (voz ~-20/-25 dB → -30/-35 dB).

- 3) ffmpeg -i [video.mp4] -af "silencedetect=noise=-30dB:d=0.5" -f null - 2>&1 | grep silence_

- 4) Concatena SÓLO los tramos a conservar con ffmpeg concat (audio+video en sincronía).

Recorta solo aire muerto real (>~0.5s). No cortes cada coma (saltos sobre el rostro). Si no hay silencios largos, no recortar es correcto.

FASE 3 · CAPTIONS Y TARJETAS DE MARCA

Un solo HTML: video + tramos de captions + 4-6 tarjetas + una línea GSAP pausada (sin setTimeout, sin bucles infinitos).

Tokens: --accent:#FF4D00; --bg:#0A0C0F; --win:#58e58c; --cost:#e5484d;

Captions: 1-3 palabras por tramo; muestra tramo.inicio-0.04 hasta el inicio del SIGUIENTE tramo -0.04.

Tarjetas: gancho, chips de modelo/feature, antes/después (rojo/verde), costo (4 facturas → 1), insignia "sin bloqueo", CTA ("comenta [palabra]").

CRÍTICO: mantén cada tarjeta lejos del rostro (bajo la etiqueta de nombre, sobre la barbilla, o tercio inferior). Haz las tarjetas GRANDES (~2.5x en 1080) y BIEN SEPARADAS.

FASE 4 · DISEÑO DE SONIDO (GRATIS)

pop.wav → ffmpeg -f lavfi -i "sine=frequency=1200:duration=0.15:beep_factor=2" pop.wav

tick.wav → ffmpeg -f lavfi -i "sine=frequency=800:duration=0.06:s=44100" tick.wav

whoosh.wav → `ffmpeg -f lavfi -i "sine=frequency=300:duration=1" -af "afade=t=in:st=0:d=0.3,afade=t=out:st=0.6:d=0.4" whoosh.wav`
Coloca los sonidos en palabras clave a bajo volumen (~0.3). La voz es la protagonista. Cama musical opcional a -18/-20 dB bajo la voz. Sin ElevenLabs, todo local.

FASE 5 · QA, RENDER Y ENTREGA

```
npx hyperframes lint [carpeta]
npx hyperframes snapshot [carpeta] --at 2.7,12.2,27.8 # y MIRA los frames
npx hyperframes render [carpeta] -o output.mp4 --fps 30
ffprobe -v error -show_entries format=duration output.mp4 # verifica el ARCHIVO, no el log
Guarda una copia FINAL y una EDITABLE.
```

OBSTÁCULOS (léelos antes de renderizar)

- Video congelado: cada elemento de video/audio necesita id único.
- Congelado al mover/buscar: re-codifica con keyframe cada segundo (`-g 30 -keyint_min 30`).
- Elemento invisible al aparecer: define el estado oculto en CSS, no en un set de GSAP en `t=0`.
- Captions amontonados: termina cada tramo al inicio del siguiente `-0.04s`.
- Overlay sobre el rostro: mantén las tarjetas en la banda clara.
- Overlays pequeños/amontonados: ~2.5x en 1080 y buenas separaciones.
- Duración incorrecta: verifica con `ffprobe` sobre el ARCHIVO.
- Captions por palabra: usa `whisper -ojf` y une los tokens.

EL ATAJO

Pide a Hermes que guarde estas cinco fases como una skill. Luego solo di:
"ejecuta el playbook de edición de video en [nuevo-video.mp4] con acento #FF4D00 y haz una copia final + editable"

PEGA TODO EL PLAYBOOK

COPIA Y PEGA TODO EL PLAYBOOK DESDE AQUÍ Y MÁS ABAJO

→ Selecciona todo lo de abajo, cópialo y pégalo en tu agente de Hermes. Ese es todo el método en un solo bloque.

CÓMO EJECUTARLO

Tú no ejecutas los comandos. Tu agente de Hermes los ejecuta. Tú aportas tres cosas: el video en crudo, un color de acento hex (o "usa naranja de marca" si quieres) y el gusto. El agente hace el resto en cinco fases.

PROMPT INICIAL (pégalo tal cual)

Aquí hay un video en crudo hablando a cámara ([video.mp4]) y quiero una edición para redes vertical. Sigue el Playbook de Edición de Video de Hermes:

- 1) transcribe la voz con tiempos por palabra (whisper local, gratis),
- 2) detecta y elimina los huecos largos de silencio,
- 3) añade captions grandes sincronizados + 4-6 tarjetas de marca (acento hex #FF4D00, fondo oscuro #0A0C0F, verde #58e58c para logros, rojo #e5484d para costos),
- 4) añade un kit de sonido gratis + una cama musical sutil (todo generado localmente, sin herramientas de pago),
- 5) renderiza un MP4 1080x1920 y guarda una copia FINAL y una EDITABLE.

Mantén cada tarjeta lejos de mi rostro. Lint, snapshot de hitos clave y MIRA, corrige lo que esté roto y luego renderiza. Pregúntame solo por el gusto del diseño y la palabra del gancho. Entrega un borrador revisable.

EL STACK (todo gratis)

- Hermes: el agente que ejecuta el playbook (app hermes)
- ffmpeg / ffprobe: sondeo, silencios, recortes, audio, verificación (brew install ffmpeg)
- whisper.cpp: transcripción local GRATIS con tiempos por palabra, sin API (brew install whisper-cpp, modelo ggml-small)
- HyperFrames: render desde HTML (npx hyperframes, sin clave)
- GSAP: motor de animación (empaquetado local, nunca CDN)
- ffmpeg SFX: genera el kit de sonido gratis, sin servicio

FASE 1 · SONDEO Y TRANSCRIPCIÓN

```
ffprobe -v error -show_entries format=duration -show_entries stream=codec_name,width,height,r_frame_rate [video.mp4]
```

```
ffmpeg -i [video.mp4] -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav
```

```
whisper-cli -m ggml-small.bin -f audio.wav -ojf -otxt -of transcript
```

El JSON -ojf da tiempos por palabra; une los tokens de sub-palabra en palabras completas.

FASE 2 · DETECCIÓN Y ELIMINACIÓN DE SILENCIOS

- 1) Revisa la salud del audio primero:

```
ffmpeg -i [video.mp4] -af "volumedetect" -vn -f null - 2>&1 | grep -E "mean_volume|max_volume"
```

Si mean_volume está por debajo de ~-60 dB, el audio está vacío - no recortes nada y avisa.

- 2) Elige el umbral ~10 dB por encima del mean_volume (voz ~-20/-25 dB → -30/-35 dB).

```
3) ffmpeg -i [video.mp4] -af "silencedetect=noise=-30dB:d=0.5" -f null - 2>&1 | grep silence_
```

- 4) Concatena SÓLO los tramos a conservar con ffmpeg concat (audio+video en sincronía).

Recorta solo aire muerto real (>=~0.5s). No cortes cada coma (saltos sobre el rostro). Si no hay silencios largos, no recortar es correcto.

FASE 3 · CAPTIONS Y TARJETAS DE MARCA

Un solo HTML: video + tramos de captions + 4-6 tarjetas + una línea GSAP pausada (sin setTimeout, sin bucles infinitos).

Tokens: --accent:#FF4D00; --bg:#0A0C0F; --win:#58e58c; --cost:#e5484d;

Captions: 1-3 palabras por tramo; muestra tramo.inicio-0.04 hasta el inicio del SIGUIENTE tramo -0.04.

Tarjetas: gancho, chips de modelo/feature, antes/después (rojo/verde), costo (4 facturas → 1), insignia "sin bloqueo", CTA ("comenta [palabra]").

CRÍTICO: mantén cada tarjeta lejos del rostro (bajo la etiqueta de nombre, sobre la barbilla, o tercio inferior). Haz las tarjetas GRANDES (~2.5x en 1080) y BIEN SEPARADAS.

FASE 4 · DISEÑO DE SONIDO (GRATIS)

```
pop.wav → ffmpeg -f lavfi -i "sine=frequency=1200:duration=0.15:beep_factor=2" pop.wav
```

```
tick.wav → ffmpeg -f lavfi -i "sine=frequency=800:duration=0.06:s=44100" tick.wav
```

```
whoosh.wav → ffmpeg -f lavfi -i "sine=frequency=300:duration=1" -af
```

```
"afade=t=in:st=0:d=0.3,afade=t=out:st=0.6:d=0.4" whoosh.wav
```

Coloca los sonidos en palabras clave a bajo volumen (~0.3). La voz es la protagonista. Cama musical opcional a -18/-20 dB bajo la voz. Sin ElevenLabs, todo local.

PEGA ESTO PARA EMPEZAR

PROMPT INICIAL

Aquí hay un video en crudo hablando a cámara ([video.mp4]) y quiero una edición para redes vertical. Sigue el Playbook de Edición de Video de Hermes: 1) transcribe la voz con tiempos por palabra (whisper local, gratis), 2) detecta y elimina los huecos largos de silencio, 3) añade captions grandes sincronizados + 4-6 tarjetas de marca (acento hex #FF4D00, fondo oscuro #0A0C0F, verde #58e58c para logros, rojo #e5484d para costos), 4) añade un kit de sonido gratis + una cama musical sutil (todo generado localmente, sin herramientas de pago), 5) renderiza un MP4 1080x1920 y guarda una copia FINAL y una EDITABLE. Mantén cada tarjeta lejos de mi rostro. Lint, snapshot de hitos clave y MIRA, corrige lo que esté roto y luego renderiza. Pregúntame solo por el gusto del diseño y la palabra del gancho. Entrega un borrador revisable.

Ese es todo el inicio. Las cinco fases de abajo son lo que el agente realmente hace — cada una es un paso comprobable y repetible.

CADA HERRAMIENTA + DÓNDE CONSEGUIRLA · TODAS GRATIS

EL STACK

TOOL	WHAT IT DOES	GET IT
Hermes	El agente de IA que ejecuta este playbook completo de principio a fin	hermes (app)
ffmpeg / ffprobe	Sondeo, detección de silencios, recortes, audio y verificación de salida	brew install ffmpeg
whisper.cpp	Transcriptor de voz local GRATIS con tiempos por palabra (sin API)	brew install whisper-cpp (modelo ggml-small)
HyperFrames	Renderiza video desde HTML: captions, GSAP, snapshot, render	npx hyperframes (sin clave)
GSAP	Motor de animación dentro de la composición	empaquetado localmente, nunca CDN
ffmpeg SFX	Genera el kit de sonido (pop, whoosh, ticks) gratis	incluido en ffmpeg, sin servicio

Cada herramienta es gratuita y corre localmente. Nada sale de tu máquina salvo el MP4 final. Si necesita una clave de pago, no está en este playbook.

FASE 1 · SONDEO Y TRANSCRIPCIÓN

Conoce tu video, obtén tiempos por palabra

Dos cosas dirigen toda la edición: la duración exacta y el tiempo exacto de cada palabra. Ambas son gratuitas.

```
# duración + resolución + fps
ffprobe -v error -show_entries format=duration \
  -show_entries stream=codec_name,width,height,r_frame_rate source.mp4

# un audio mono limpio de 16kHz para la transcripción
ffmpeg -i source.mp4 -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav

# transcripción local GRATIS CON tiempos por palabra (sin API)
whisper-cli -m ggml-small.bin -f audio.wav -ojf -otxt -of transcript
```

El JSON `-ojf` da tiempos de palabra por token dentro de cada segmento — eso permite que los captions caigan en la palabra exacta. (Algunas versiones de whisper dan tiempos por token; tu agente puede unir tokens de sub-palabra en palabras completas.)

FASE 2 · DETECCIÓN Y ELIMINACIÓN DE SILENCIOS

Corta el aire muerto

Detecta dónde la voz está en silencio y reensambla solo los segmentos hablados para que el video quede compacto — sin huecos largos.

```
# encuentra huecos de silencio (reporta donde la voz pausa >=0.5s)
ffmpeg -i source.mp4 -af "volumedetect" -vn -f null - 2>&1 | grep -E "mean_volume|max_volume"
# ^ PRIMERO: lee el mean_volume. Elige tu umbral de silencio aprox.
#   10 dB por encima (la voz suele estar en -20 a -25 dB → usa -30 a -35 dB)

ffmpeg -i source.mp4 -af "silencedetect=noise=-30dB:d=0.5" -f null - 2>&1 \
  | grep silence_

# luego concatena SOLO los tramos a conservar (el agente los une
# con el filtro concat de ffmpeg para que audio+video queden sincronizados)
```

¡Nunca a cortara como pausas cortas naturales. Procura solo el aire muerto real (un tiempo o más) — no cortes cada coma o brevedad natural sobre el resto.

Una fuente con silencios y con corrección de color (mean volume alrededor de -30 dB) NO tiene voz audible, así que CUALQUIERA umbral la marcará toda como "silencio". Revisa mean volume primero: si está por debajo de -30 dB, el audio del archivo está efectivamente vacío — no recortes nada y detiene el proceso.

Si un video no tiene silencios largos (mucha energía, habla constante), omite el corte por completo — no recortar nada también es correcto. Con voz real (media ~-22 dB), -30 dB encuentra solo el aire muerto real.

Construye la composición

Un solo archivo HTML contiene el video + tramos de captions + 4-6 tarjetas de marca + una línea de tiempo GSAP pausada. Mantén las tarjetas lejos del rostro (bajo la etiqueta de nombre, sobre la barbilla / o en el tercio inferior).

```
# tokens de acento que hereda cada tarjeta
:root { --accent:#FF4D00; --bg:#0A0C0F; --win:#58e58c; --cost:#e5484d; }

# regla de captions: 1-3 palabras por tramo; muestra chunk.start-0.04
# hasta el inicio del SIGUIENTE tramo -0.04 (si no, se amontonan)
```

Un set mínimo de tarjetas probado

- **Tarjeta gancho** — primera línea, palabra grande en mayúsculas
- **Chips de modelo/feature** — lista los nombres clave
- **Antes / Después** — rojo vs verde
- **Tarjeta de costo** — 4 facturas → 1, de rojo a verde
- **Insignia "Sin bloqueo"** — contorno verde
- **CTA** — "comenta [palabra]" + línea de guía

Cada número recibe un contador. Cada lista aparece sincronizada con las palabras. Reglas que el agente puede seguir siempre.

FASE 4 · DISEÑO DE SONIDO (GRATIS)

Un kit de 6 sonidos, generado por ffmpeg

Sin ElevenLabs. ffmpeg sintetiza pequeños sonidos libres de regalías; luego los colocas en palabras clave — un whoosh al entrar una tarjeta, ticks bajo un contador, un pop en un chip.

```
# genera el kit con ffmpeg (gratis, local) – todos los comandos probados
# pop: un beep corto y contundente
ffmpeg -f lavfi -i "sine=frequency=1200:duration=0.15:beep_factor=2" pop.wav
# tick: un clic agudo muy corto
ffmpeg -f lavfi -i "sine=frequency=800:duration=0.06:s=44100" tick.wav
# whoosh: un barrido que sube y baja (fade in/out = la forma "whoosh")
ffmpeg -f lavfi -i "sine=frequency=300:duration=1" -af \
    "afade=t=in:st=0:d=0.3,afade=t=out:st=0.6:d=0.4" whoosh.wav
```

Para una cama musical, genera también un pad ambiental suave con ffmpeg. O añade cualquier cama libre de regalías que tengas. Mantenla -18 a -20 dB bajo la voz para que el habla quede clara.

Cantidad del balanceo: coloca los sonidos a bajo volumen (-0.3). La voz es la protagonista, los efectos deben caer y desaparecer bajo ella.

Mira los píxeles. Luego publica.

Cada error se detecta mirando los fotogramas, nunca releendo el código. Lint, snapshot, MIRA, corrige, renderiza, y luego verifica el archivo.

```
# lint + snapshot de hitos clave + render
npx hyperframes lint
npx hyperframes snapshot --at 2.7,12.2,27.8
npx hyperframes render -o output.mp4 --fps 30

# verifica el ARCHIVO (nunca confíes en el log del render)
ffprobe -v error -show_entries format=duration output.mp4
```

Guarda una copia FINAL y una EDITABLE — para que cualquier ajuste sea un regenerar, no un rehacer.

LÉELO ANTES DE TU PRIMER RENDER

Los obstáculos

SYMPTOM	FIX
El video se renderiza como un fotograma congelado	Cada elemento de video/audio necesita un id único.
Fotograma congelado cuando el video se mueve/busca	Re-codifica con un keyframe cada segundo (-g 30 -keyint_min 30).
Elemento invisible en su primera aparición	Define el estado oculto inicial en CSS, no en un set de GSAP en el tiempo 0.
Dos captions amontonados	Termina cada tramo al inicio del siguiente menos 0.04s.
Overlays cubriendo el rostro del hablante	Mantén las tarjetas en la banda clara — bajo la etiqueta de nombre, sobre la barbilla, o en el tercio inferior.
Overlays demasiado pequeños / amontonados	Dimensiona a ~2.5x en una pantalla 1080 y deja grandes separaciones entre elementos.
Duración incorrecta	Verifica con ffprobe sobre el ARCHIVO, no el log del render.
Necesitas captions por palabra	Usa whisper -ojf y une los tokens de palabras para tiempos precisos.

EL ATAJO · UNA SKILL INSTALADA

Guárdala como skill de Hermes

Pide a Hermes que guarde este playbook de cinco fases como una **skill** reutilizable. La próxima vez solo dices:

```
ejecuta el playbook de edición de video en [nuevo-video.mp4] con acento #FF4D00  
y haz una copia final + editable
```

POR QUÉ GRATIS Y LOCAL

Sin cuotas mensuales, sin límites de crédito, sin gestión de claves. ffmpeg + whisper + HyperFrames son de código abierto y corren en tu máquina.

POR QUÉ FUNCIONA SIEMPRE

Cada paso es comprobable y determinista: lint, snapshot de frames, MIRA, corrige. El agente nunca renderiza a ciegas.